

Multilevel Hypergraph Partitioning

粗化算法 EC、HEC、MHEC 与最粗层初始划分

依据 Karypis et al., IEEE TVLSI, 1999

学习日期：2026 年 9 月 15 日

今日目标

今天从“知道三阶段”进入“能解释第一阶段为什么这样设计”。完成后，你应当能够：

1. 写出粗化必须同时满足的两个目标；
2. 手算 EC 中两个顶点的连接强度，并完成一轮匹配；
3. 区分 EC、HEC、MHEC 的聚合单位、优点和失效模式；
4. 解释粗化时顶点权重、超边合并与自环超边如何更新；
5. 说明为什么最粗层不求精确最优，以及为何要保留多个候选解。

建议学习时长：100 分钟。本日范围是论文第 II-A、II-B 节；FM/HER 的 gain 更新留到第三天。

目录

1	五分钟复盘：粗化不是单纯“减少点数”	3
2	统一视角：一次收缩到底更新什么	3
2.1	加权超图与聚合映射	3
3	EC：把“共享超边强度”转成成对匹配	4
3.1	EC 的评分：公式逐项拆解	4
3.2	为什么分母是 $ e - 1$?	5
3.3	完整比较例子：EC 为什么选择 b ?	5
3.4	手算例子：一轮 EC	6
4	HEC：直接按超边聚合	8
4.1	为什么 EC 对真正的大超边不够强?	8
4.2	HEC 的收益与代价	8
5	MHEC：修补 HEC 留下的“半缩超边”	9
6	三种粗化策略的统一比较	9

7 何时停止粗化？最粗层怎样求初始划分？	10
7.1 什么时候停止？	10
7.2 平衡的是顶点权重，不是粗顶点个数	11
7.3 方法一：随机平衡二分	11
7.4 方法二：随机起点的区域生长	12
7.5 为什么生成之后还要运行 FM？	12
7.6 为什么不在 H_m 上求精确最优？	12
7.7 为什么要保留多个候选？怎样剪枝？	13

1 五分钟复盘：粗化不是单纯“减少点数”

第一天的主线是

$$H_0 \xrightarrow{\text{coarsen}} H_1 \rightarrow \dots \rightarrow H_m \xrightarrow{\text{initial partition}} P_m \xrightarrow{\text{project+refine}} P_0.$$

今天只放大前半段。论文明确给粗化两项任务：

1. 构造足够小的 H_m ，使它的好二分投影回原超图后仍有潜力成为好解；
2. 逐层缩小大超边，使 KL/FM 类局部细化在后续层次更容易发挥作用。

核心判断

粗化的质量不等于“缩得最快”。真正目标是：**在显著减小规模的同时，尽量不压死有希望的切分边界**。错误聚合一旦把本应分居两侧的顶点锁进同一粗顶点，粗层就无法表达那条候选切分。

2 统一视角：一次收缩到底更新什么

2.1 加权超图与聚合映射

设当前层为 $H_i = (V_i, E_i)$ 。每个顶点 v 有权重 $w(v)$ ，每条超边 e 有权重 $w(e)$ 。原始无权网表可令所有权重为 1。粗化选择若干互不重叠的顶点组

$$C_1, C_2, \dots, C_r, \quad C_j \subseteq V_i,$$

并把每组变成 H_{i+1} 的一个粗顶点 c_j 。其权重为

$$w(c_j) = \sum_{v \in C_j} w(v).$$

每条超边的端点通过映射 $\pi : V_i \rightarrow V_{i+1}$ 更新：

$$e' = \{\pi(v) : v \in e\}.$$

更新后需要做两种规范化：

- 若不同超边得到相同端点集合，则**合并平行超边**，权重相加；
- 若 $|e'| = 1$ ，它已完全落在一个粗顶点内部，永远不会被切，故**删除自环超边**。

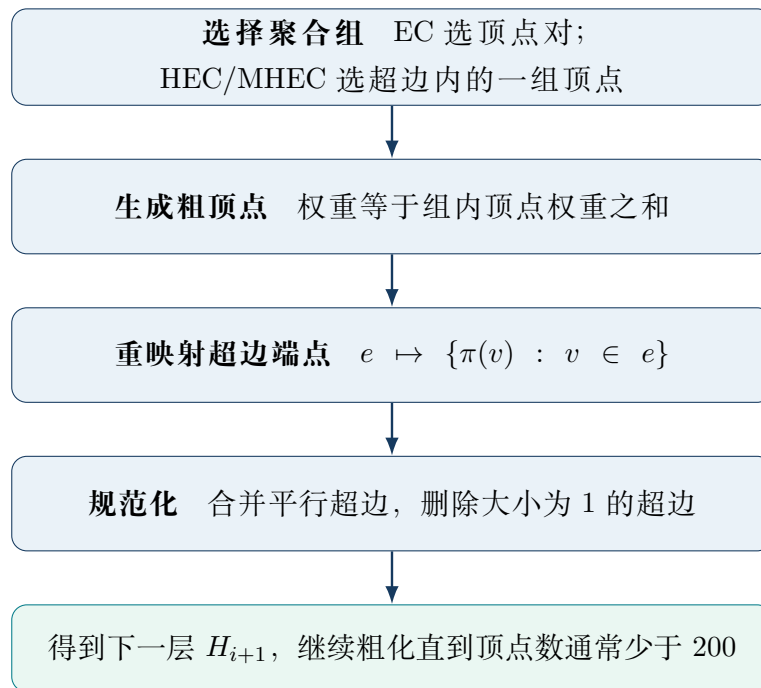


图 1: 一次粗化层的纵向数据变换

3 EC: 把“共享超边强度”转成成对匹配

3.1 EC 的评分: 公式逐项拆解

EC (edge coarsening) 要回答的问题是: 对当前未匹配顶点 u , 哪个未匹配顶点 v 最适合与它收缩? 论文把每条超边看作一个隐式 clique, 并定义顶点对的连接强度为

$$s(u, v) = \sum_{\substack{e \in E \\ u, v \in e}} \frac{w(e)}{|e| - 1}$$

各符号含义如下:

- u, v : 正在判断是否应当匹配的两个顶点;
- e : 同时包含 u 和 v 的超边; 不同时包含二者的超边不参加求和;
- $w(e)$: 超边权重; 无权超图通常令 $w(e) = 1$;
- $|e|$: 超边包含的顶点数;
- $s(u, v)$: 二者的总连接强度, 数值越大, 越优先收缩。

先把公式读成一句话

$s(u, v)$ 等于: 把所有同时包含 u, v 的超边找出来, 再把每条超边提供的 $w(e)/(|e| - 1)$ 贡献加起来。因此, 多共享、高权重、小超边都会增大连接强度。

3.2 为什么分母是 $|e| - 1$?

若 $e = \{a, b, c, d\}$, 对顶点 a 来说, 它在这条超边中有 $|e| - 1 = 3$ 个伙伴: b, c, d 。单位权重超边对每一对分别贡献 $1/3$, 所以从 a 分配出去的总强度为

$$\frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1.$$

一般地, 大小为 k 的单位权重超边中, 一个顶点有 $k - 1$ 个伙伴, 故

$$(k - 1) \times \frac{1}{k - 1} = 1.$$

这是一种归一化: 避免大超边仅仅因为顶点多, 就人为制造大量强连接。

比较两个极端例子:

$$e_s = \{a, b\} : \frac{1}{|e_s| - 1} = 1,$$

$$e_L = \{a, b, c, d, e, f, g, h, i, j\} : \frac{1}{|e_L| - 1} = \frac{1}{9}.$$

二元超边只连接 a, b , 关系很直接; 十元超边只说明 a, b 同属一个大群体, 证据较弱。因此

小超边中的共同出现, 比大超边中的共同出现更能支持成对收缩。

3.3 完整比较例子: EC 为什么选择 b ?

设所有超边权重为 1:

$$e_1 = \{a, b, c\}, \quad e_2 = \{a, b\}, \quad e_3 = \{a, c, d, e\}.$$

对候选 b, c, d 分别计算:

$$\begin{aligned} s(a, b) &= \frac{1}{|e_1| - 1} + \frac{1}{|e_2| - 1} = \frac{1}{2} + 1 = \frac{3}{2}, \\ s(a, c) &= \frac{1}{|e_1| - 1} + \frac{1}{|e_3| - 1} = \frac{1}{2} + \frac{1}{3} = \frac{5}{6}, \\ s(a, d) &= \frac{1}{|e_3| - 1} = \frac{1}{3}. \end{aligned}$$

于是

$$s(a, b) > s(a, c) > s(a, d),$$

若三者均未匹配, EC 在访问 a 时优先选择 b , 形成匹配 (a, b) 。

若超边带权, 例如 $w(e_1) = 4, w(e_2) = 2$, 则

$$s(a, b) = \frac{4}{3 - 1} + \frac{2}{2 - 1} = 2 + 2 = 4.$$

可以把每条超边的贡献理解为

$$\underbrace{w(e)}_{\text{超边有多重要}} \times \underbrace{\frac{1}{|e| - 1}}_{\text{关系有多专一}}.$$

得到的是**重边极大匹配 (heavy-edge maximal matching)**: 算法按随机顺序贪心地选取当前最强且不冲突的顶点对。它保证最后不能再加入新的匹配对 (maximal), 但不保证所有匹配对的总强度达到全局最大 (maximum-weight)。论文选择前者, 是因为粗化必须足够快速。

三个易错点

1. 求和范围只包括**同时包含** u, v 的超边；
2. 分母是超边大小减一，即 $|e| - 1$ ，不是 $|e|$ ；
3. $s(u, v)$ 只决定当前贪心选择，最终匹配仍会受到随机访问顺序和“顶点不能重复匹配”的影响。

3.4 手算例子：一轮 EC

考虑顶点 $V = \{a, b, c, d, e, f\}$ ，所有超边权重均为 1：

$$e_1 = \{a, b, c\}, \quad e_2 = \{a, b\}, \quad e_3 = \{c, d, e\}, \quad e_4 = \{d, e\}, \quad e_5 = \{e, f\}.$$

为便于手算，采用归一化贡献 $1/(|e| - 1)$ 。则

$$s(a, b) = \frac{1}{3-1} + \frac{1}{2-1} = \frac{3}{2}, \quad s(a, c) = \frac{1}{2},$$

$$s(d, e) = \frac{1}{2} + 1 = \frac{3}{2}, \quad s(e, f) = 1.$$

若随机访问顺序从 a 开始，则 a 优先与 b 匹配；随后若访问 d ，则 d 与 e 匹配； c, f 暂时单独。粗顶点为

$$A = \{a, b\}, \quad C = \{c\}, \quad D = \{d, e\}, \quad F = \{f\},$$

权重分别为 2, 1, 2, 1。重映射后：

$$e'_1 = \{A, C\}, \quad e'_2 = \{A\}, \quad e'_3 = \{C, D\}, \quad e'_4 = \{D\}, \quad e'_5 = \{D, F\}.$$

其中 e'_2, e'_4 为大小 1 的超边，删除。于是 6 个点缩为 4 个点，同时两条网已完全内化。

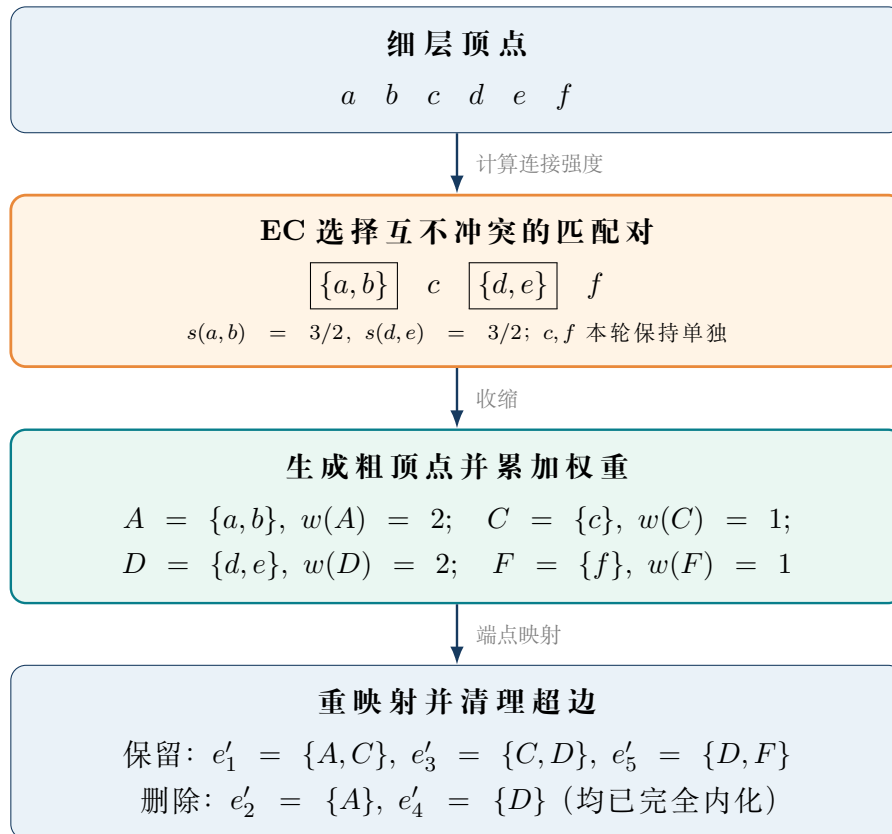


图 2: EC 手算例子的纵向图解：匹配、收缩、重映射与删除内化超边

检查点：匹配为什么依赖随机访问顺序？

若某顶点先与一个“次优但尚可”的邻点匹配，后续顶点就不能再使用它。因此 EC 是便宜的随机贪心启发式，而不是确定的全局最优匹配。随机性会产生不同层次结构，也为多次运行提供搜索多样性。

4 HEC：直接按超边聚合

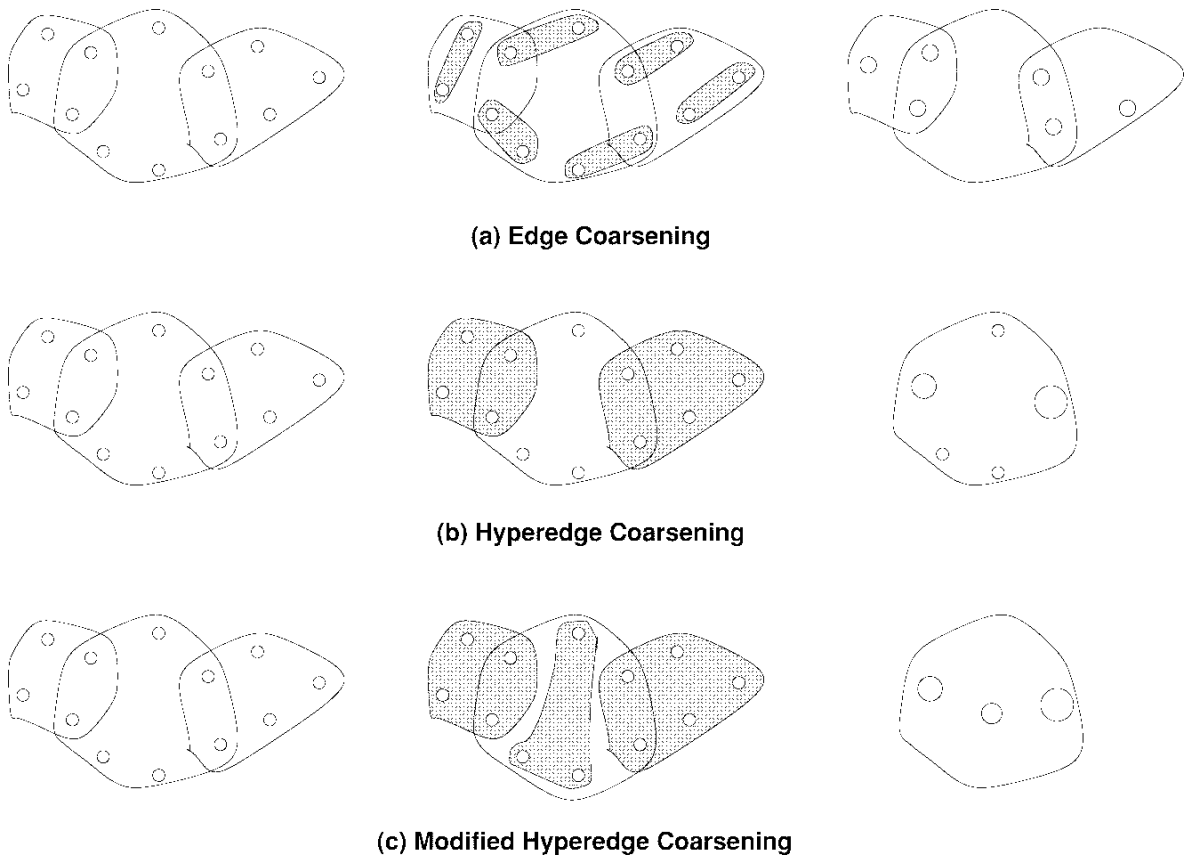


图 3: EC、HEC 与 MHEC 诱导的不同粗化方式 (论文 Fig. 2) (原论文图, 供学习引用)

4.1 为什么 EC 对真正的大超边不够强？

EC 每次只合并一对顶点。对于 $e = \{v_1, \dots, v_k\}$, 即使其中一对被合并, 粗化后的 e 往往仍有 $k-1$ 个端点; 只有大小为 2 的超边在一对顶点合并后会立即完全内化。因而 EC 虽能减点, 却未必快速减小大超边。

HEC (hyperedge coarsening) 改为: 从互不冲突的候选超边中选择一条, 把该超边包含的全部顶点收缩为一个粗顶点。论文倾向优先选择**权重大且规模小**的超边。直觉是:

- 权重大: 保住这条网“不被切”更有价值;
- 规模小: 把少量强相关点锁在一起, 误伤潜在边界的风险较低;
- 整条收缩: 该超边立即内化, 超边总规模下降比 EC 更快。

4.2 HEC 的收益与代价

维度	收益	风险
收缩强度	一次可合并两个以上顶点, 减点快	聚合组可能过大
超边规模	被选超边立即内化	未被选超边可能几乎不缩
解空间	强相关模块整体移动	若选错, 会排除某些切分
顶点权重	更快形成高层模块	粗顶点权重可能高度不均

5 MHEC: 修补 HEC 留下的“半缩超边”

HEC 要求选中的超边互不重叠，否则一个顶点会同时被分进多个粗顶点。因此第一遍结束后，很多未被选超边只收缩了一部分，甚至完全没收缩。MHEC (modified HEC) 的思想是：

1. 第一遍像 HEC 一样，优先整条收缩合适的独立超边；
2. 第二遍重新看未充分处理的超边；
3. 在不破坏已有聚合的前提下，把剩余未归组顶点附加到相关聚合，或把剩余子集进一步组成粗顶点。

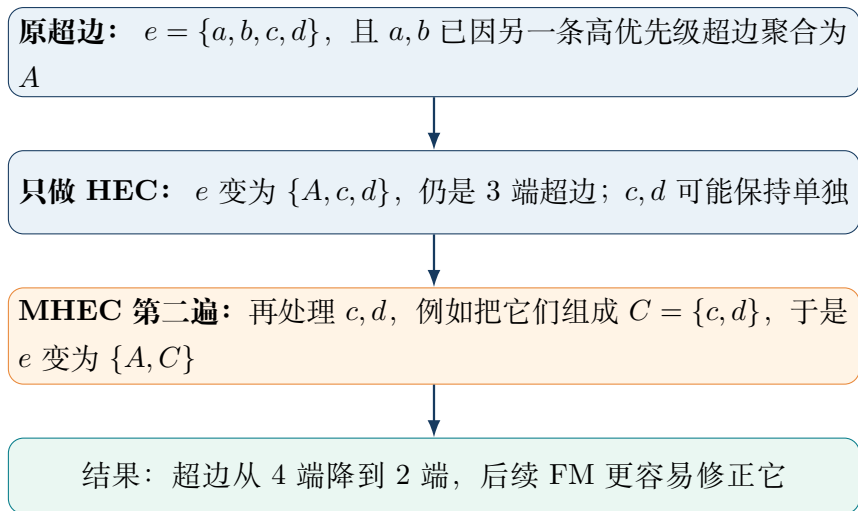


图 4: MHEC 的“第二遍补缩”直觉示例

三种方法的一句话区分

EC 问：“这个顶点最该和哪个顶点成对？”；**HEC** 问：“哪条超边值得整体变成一个模块？”；**MHEC** 再问：“第一遍之后，哪些超边里的剩余顶点还应继续归组？”

6 三种粗化策略的统一比较

方法	基本聚合单位	收缩强度	主要优势	主要弱点
EC	顶点对	温和	粗顶点权重较均匀；实现简单	大超边缩得慢
HEC	整条独立超边	最强	快速内化超边，直接保持超图结构	聚合过大；遗漏重叠超边
MHEC	整条超边 + 剩余子集	强且更充分	兼顾强粗化与减少半缩超边	决策更复杂，仍属启发式

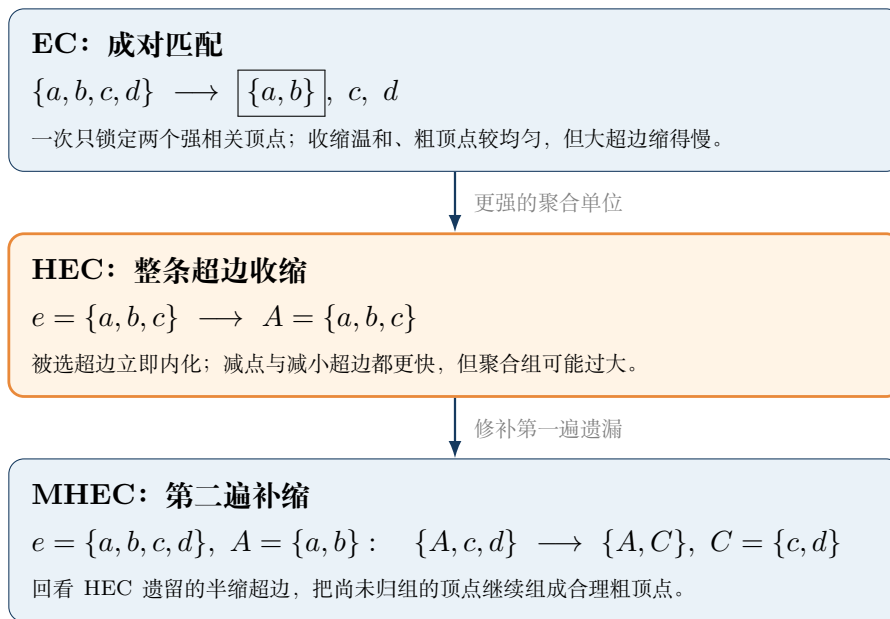


图 5: EC、HEC、MHEC 的纵向视觉对照：成对匹配、整边收缩与第二遍补缩

选择策略时不存在“任何数据上都最优”的证明。论文通过实验比较具体变体；我们此处应掌握的是设计权衡：

收缩速度 $\longleftrightarrow$ 粗顶点平衡 $\longleftrightarrow$ 保留潜在切分自由度。

7 何时停止粗化？最粗层怎样求初始划分？

7.1 什么时候停止？

论文把顶点数缩到通常少于 200 后停止。此时问题仍然不平凡，但已经足够小，可以用便宜的启发式反复生成初始二分：

- **随机平衡划分**：按顶点权重把粗顶点分到两侧；
- **区域生长**：从随机顶点出发，用 BFS 扩张一侧直到接近平衡；
- 对所得候选运行 FM，提高最粗层 cut 质量。

初始划分阶段要同时满足两个要求：

满足平衡约束 + cut 尽可能小。

注意最粗层是“模块级”布局：把粗顶点 $A = \{a, b, c, d\}$ 放入某一侧，等价于暂时把 a, b, c, d 四个原始顶点一起放在那里。

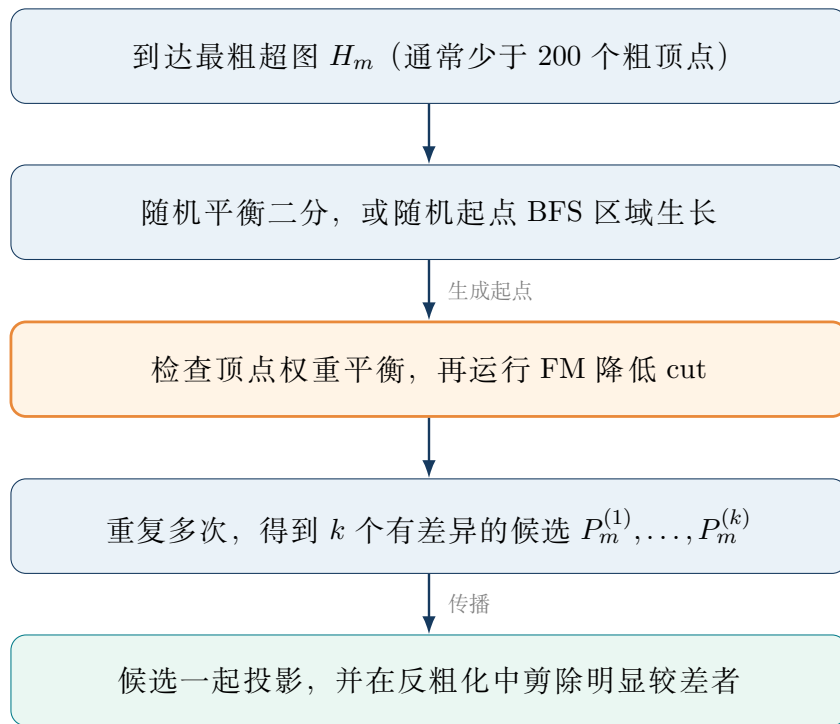


图 6: 最粗层初始划分的纵向流程: 快速生成、FM 改善、多候选传播

7.2 平衡的是顶点权重, 不是粗顶点个数

粗顶点代表不同数量的原始顶点, 所以平衡约束比较的是权重之和。设粗顶点权重为

$$w(A) = 4, w(B) = 3, w(C) = 2, w(D) = 2, w(E) = 1, w(F) = 1,$$

则总权重为

$$W = 4 + 3 + 2 + 2 + 1 + 1 = 13,$$

理想目标是每侧约 $13/2 = 6.5$ 。例如

$$V_m^0 = \{A, C, E\}, \quad W(V_m^0) = 7, \quad V_m^1 = \{B, D, F\}, \quad W(V_m^1) = 6.$$

虽然两侧粗顶点个数都是 3, 但换成

$$V_m^0 = \{A, B, C\}, \quad W(V_m^0) = 9, \quad V_m^1 = \{D, E, F\}, \quad W(V_m^1) = 4$$

同样“每侧三个”, 实际负载却严重失衡。因此:

平衡看的是 $W(V_m^i)$, 不是粗顶点的个数.

7.3 方法一: 随机平衡二分

基本做法是: 随机打乱粗顶点, 依次把它们分配到当前的某一侧, 使两侧权重尽量接近, 最后计算 cut。以前面的权重为例, 可能得到

$$V_m^0 = \{D, F, B\}, \quad W = 6, \quad V_m^1 = \{A, C, E\}, \quad W = 7,$$

对应 cut

$$C(P_m) = \sum_{\substack{e \in E_m \\ e \text{ 跨越两侧}}} w(e).$$

随机平衡二分只保证初步平衡，不保证 cut 小，所以随后必须运行 FM。

7.4 方法二：随机起点的区域生长

第二种方法从随机顶点出发，沿超图用 BFS 向外扩张，直到一侧权重接近总权重的一半。设

$$e_1 = \{A, B, C\}, \quad e_2 = \{C, D\}, \quad e_3 = \{D, E, F\},$$

所有权重为 1，随机起点为 A 。则先放入 A ，再经 e_1 扩张到 B, C ，得到

$$V_m^0 = \{A, B, C\}, \quad V_m^1 = \{D, E, F\}.$$

此时只有 $e_2 = \{C, D\}$ 被切，故 $C(P_m) = 1$ 。区域生长利用了超图的局部结构，因此通常比纯随机分配更容易得到一个连通的初始区域。

带权时要按累计权重判断停止。设总权重 $W = 20$ ，BFS 依次遇到权重为

$$w(A) = 3, \quad w(B) = 2, \quad w(C) = 4, \quad w(D) = 3$$

的顶点，累计为

$$3 \rightarrow 5 \rightarrow 9 \rightarrow 12.$$

若允许 45%–55% 平衡，则每侧允许范围为 9 到 11，因此应停在 $W = 9$ 处：

$$V_m^0 = \{A, B, C\}, \quad V_m^1 = \{D, E, F\}$$

(此时 D 不加入)。

7.5 为什么生成之后还要运行 FM？

初始划分只是起点。若当前 cut 为 4，而把粗顶点 C 移到另一侧后仍满足平衡、cut 降为 2，则

$$g(C) = 4 - 2 = 2,$$

FM 会接受这次移动。所以完整流程是

随机平衡二分或区域生长 $\rightarrow$ FM 细化 $\rightarrow$ 一个初始候选.

7.6 为什么不在 H_m 上求精确最优？

1. **搜索空间依然很大：**即使只剩 100 个粗顶点，也有约 2^{100} 种二分选择，加上平衡约束仍属困难组合优化；
2. **最粗层只是近似模型：**粗化已限制其可表达的切分，其精确最优不等于 H_0 的最优；
3. **反粗化会大幅修改划分：**投影恢复大量自由度，FM/HER 还会持续改变边界，把大量时间花在稍后会被显著改动的代理问题上收益不高。

7.7 为什么要保留多个候选？怎样剪枝？

两种初始划分算法都带随机性：随机顺序、随机起点，以及 FM 在相同 gain 下的不同选择，都会产生不同候选。不同候选在反粗化中的改进潜力不同，最粗层 cut 最小的那个不一定给出原图最小的最终 cut。例如：

候选	最粗层 cut	反粗化后的最终 cut
$P_m^{(1)}$	8 (最粗层最好)	13
$P_m^{(2)}$	10	11 (最终最好)
$P_m^{(3)}$	9	12

所以 hMETIS 不在最粗层只留一个解，而是让多个候选一起进入反粗化，并动态剪枝：删除当前 cut 比该层最佳值差太多的候选。设该层最佳 cut 为 $C_{\min} = 20$ ，采用 10% 阈值即保留 $C(P) \leq 22$ ：

候选	当前 cut	是否保留
$P^{(1)}$	20	保留（与最佳相同）
$P^{(2)}$	21	保留（在 10% 内）
$P^{(3)}$	22	保留（恰为阈值上限）
$P^{(4)}$	23	删除（超出 10%）

这样可以做到“粗层多探索、细层少保留”：粗层 cut 对最终结果预测不准，因而值得保留多样性；而细层顶点多、细化昂贵，此时只能保留少量候选。论文的实验设置是：最粗层生成 10 个候选，反粗化中淘汰比当前最佳 cut 差 10% 以上的候选。作者报告：由 1 个增至 10 个候选，平均 cut 改善约 3%–4%，时间约增至两倍；由 10 增至 20，额外改善不足 0.5%，说明候选数存在明显的边际收益递减。

不要混淆“多个候选”与“多次独立运行”

多个候选是在同一次多层流程的最粗层并行保留若干起点，并可在反粗化时剪枝；多次独立运行则从头重新随机粗化。两者都利用随机性，但计算共享程度不同。

第三天预告 第三天学习反粗化与细化：从一个已知二分出发，逐步计算 FM gain、bucket 选择、锁定与回滚，再解释 hMETIS 为什么使用 two-pass 与 early-exit，以及 HER 如何一次移动一组顶点来处理大超边。

来源与精度说明

本文依据用户提供的原论文第 II-A、II-B 节与 Fig. 2 整理。EC 的教学手算采用论文明确给出的隐式 clique 权重：大小为 $|e|$ 的超边贡献 $1/(|e| - 1)$ ；若粗化后超边塌缩成相同端点集合，则权重累加。手算例子用于解释算法机制，不是论文 benchmark 的复现。